a01.c | queue.c

practicalpractice > ds > queue.c > dequeue()

```c
#include <stdio.h>

int front = -1;
int rear = -1;
int queue[5];
int job;


void enqueue(){

    if(rear == 4){
        printf("Queue overflow\n");
    }
    else{
        printf("Enter job: ");
        scanf("%d", &job);

        if (front == -1){
            front = 0;
        }
        rear++;
        queue[rear] = job;
        printf("Job added successfully\n");
    }
}

void dequeue(){

    if(front == -1){
        printf("Queue underflow\n");
    }
    else{
        printf("Processed job: %d ", queue[front]);
        if(front == rear){
            front = -1;
            rear = -1;
        }
        else{
        front++;
        }
    }
}

void display(){

    int i;

    if(front == -1){
        printf("Queue is empty\n");
    }
    else{
        printf("pending jobs: ");

        for(i= front; i<= rear; i++){
            printf("%d  ", queue[i]);
        }
    }
}

int main(){
    int choice;

    do{
        printf("\nEnter your choice: ");
        scanf("%d", &choice);

```

a01.c | queue.c

practicalpractice > ds > queue.c > dequeue()

```
        printf("\n1. Enqueue\n");
        printf("2. Dequeue\n");
        printf("3. Display Queue\n");
        printf("4. Exit\n");

    switch (choice)
    {
    case 1:
        enqueue();
        break;

    case 2:
        dequeue();
        break;

    case 3:
        display();
        break;

    case 4:
        printf("Exiting program...");
        break;

    default:
        printf("Invalid choice, please choose again.");
        break;
    }
}while(choice !=4);

return 0;
}
```

Problems | Output | Debug Console | Terminal | Ports

```
siddhijha@Siddhis-MacBook-Air C prog % cd "/Users/siddhijha/Desktop/C prog/practicalpractice/ds/" && gcc queue.c -o queue && "/Users/siddhijha/Desktop/C prog/practicalpracti
ce/ds/"queue

1. Enqueue
2. Dequeue
3. Display Queue
4. Exit
Invalid choice, please choose again.
Enter your choice: 1

1. Enqueue
2. Dequeue
3. Display Queue
4. Exit
Enter job: 78
Job added successfully

Enter your choice: 3

1. Enqueue
2. Dequeue
3. Display Queue
4. Exit
pending jobs: 24  45  78
Enter your choice: 2

1. Enqueue
2. Dequeue
3. Display Queue
4. Exit
Processed job: 24
Enter your choice: 3

1. Enqueue
2. Dequeue
3. Display Queue
4. Exit
pending jobs: 45  78
```

Siddhi Jha
1262253755

ASSIGNMENT-09

Job Queue simulation Using linear Queue

Problem statement:

Queues are frequently used in computer programming, and a typical example is the creation of a job queue by an operating system. If the operating system does not use priorities, then the jobs are processed in the order they enter the system. Write a c program for simulating a job queue. Write functions to add a job and delete a job.

Objective:

1) Understand the concept of Queue and FIFO (First In, Fist Out)
2) Implemet a linear queue using an array.
3) Implement functions to add and delete jobs.
4) Simulate processing of jobs in the order they enter the system.
5) Analyze the time and space complexity of queue operations.

Implementation:

Platform: · 64-bit Open source linux or its derivatives
· Open source C programming tool like gcc/Eclipse Editor.

Test Conditions:

Test case	Input	Expected Output
TC01	Add one job	Job added successfully
TC02	Add multiple jobs	Jobs sorted in FIFO order
TC03	Delete a job	First entered job is deleted/processed
TC04	Delete from empty queue	Queue underflow message
TC05	Add jobs when queue is full	Queue Overflow message
TC06	Display pending jobs	All jobs displayed from front to rear.

FOR EDUCATIONAL USE

Pseudo-code

```
START
Initialize
    FRONT = -1
    REAR = -1
    Create QUEUE [MAX]

REPEAT
    Display Menu
    1. Add job
    2. Delete / Process job
    3. Display pending jobs
    4. EXIT

Accept choice
case 1: Add job
    IF REAR = MAX - 1 THEN
        PRINT "Queue Overflow - Cannot add more jobs"
    ELSE
        INPUT JOB

        IF FRONT = -1 THEN
            FRONT = 0
        ENDIF

        REAR = REAR + 1
        QUEUE [REAR] = JOB

        PRINT "Job added successfully"
        ENDIF
```

FOR EDUCATIONAL USE

CASE 2 : Delete / Process Job

```
IF FRONT = -1 OR FRONT > REAR THEN
    PRINT "Queue Underflow - No jobs to process"
ELSE
    JOB = QUEUE [FRONT]
    PRINT "Processing job:", JOB

    IF FRONT = REAR THEN
        FRONT = -1
        REAR = -1
    ELSE
        FRONT = FRONT + 1
    ENDIF
ENDIF
```

Case 3 : Display Pending Jobs

```
IF FRONT = -1 OR FRONT > REAR THEN
    PRINT "No Pending Jobs"
ELSE
    PRINT "Pending Jobs are :"
    FOR I = FRONT TO REAR
        PRINT QUEUE [I]
    END FOR
ENDIF
```

Case 4 :

```
PRINT "Exiting Program"
```

FOR EDUCATIONAL USE

DEFAULT :
PRINT "Invalid Choice"

UNTIL CHOICE = 4
STOP

Time complexity :

add Job (Enqueue)	O(1)
Delete / Process Job (Dequeue)	O(1)
Display Pending Jobs	O(n)
Check overflow / Underflow	O(1)

Space complexity :
O(n), where n is the max^m size of the queue.

Conclusion :
The job queue simulation demonstrates the FIFO principle linear queue. Jobs are added at the rear and processe from the front in the same order in which they ent the system.

FAQs

1) What is a Queue ?

ns : A Queue is a linear data structure in which insertion performed at the rear and deletion is performed from front. It follows the FIFO principle.

) What is FIFO ?

FIFO (First In First Out) means the element inserted first removed first. For example, in a ticket line, the person

FOR EDUCATIONAL USE

comes first is served first.

3) Why is a queue suitable for an operating-system job queue?
: A queue is suitable because jobs are processed in the same order in which they arrive. This ensures fairness & systematic execution of tasks.

4) What is Queue Overflow & Queue Underflow?
: · Overflow: Occurs when an insertion is attempted in full queue.
· Underflow: Occurs when a deletion is attempted from an empty queue.

5) What are the applications of Queue?
· CPU / job scheduling
· Printer spooling
· Ticket reservation systems.
· Call centre management
· Network packet handling
· Breadth First Search (BFS)

6) What is a Circular Queue?
: A circular queue in which the last position is connected back to the first position, forming a circle. It allows reuse a vacant spaces created after deletions.

7) Why is a Circular Queue preferred over a linear queue?
: A circular Queue utilizes memory more efficiently because freed positions can be reused. In a linear Queue, empty spaces at the front cannot be reused once the rear reaches the end.

FOR EDUCATIONAL USE

8) What principle does a circular Queue following?

Ans: A circular Queue also follows the FIFO principle. The inserted element is the first one removed.

9) What are the basic operations performed on a circular Queue?

i) Enqueue - Insert an element
ii) Dequeue - Remove an element
iii) Peek / Front - View the front element
iv) isEmpty - Check whether the queue is empty.
v) isFull - Check whether is queue is full.
vi) Display - Show all queue elements

10) What is overflow and Underflow condition in a circular Queue?

Overflow condition:

(FRONT == (REAR + 1) % MAX)

When the next position of REAR becomes FRONT, the queue

Underflow condition:

FRONT == -1

When there are no elements in the queue, the queue is